UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ciencias

Escuela Profesional de Ciencia de la Computación

Asignatura: Ingeniería de Software

Tema: **Principios de Diseño de Software: Cohesión, Separación of Concerns y Acoplamiento**

Expositor: **Américo Chulluncuy Reynoso**

Proceso de Contratación Docente 2026-I

¿Qué es la Ingeniería de Software?

- IEEE:
  - “La aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software.”
- Roger S. Pressman:
  - “La ingeniería de software es una disciplina de ingeniería que se ocupa de todos los aspectos de la producción de software.”

- ¿Cómo diseñamos software de calidad?
  - Principios fundamentales: Cohesión, Acoplamiento, Separación de Concerns

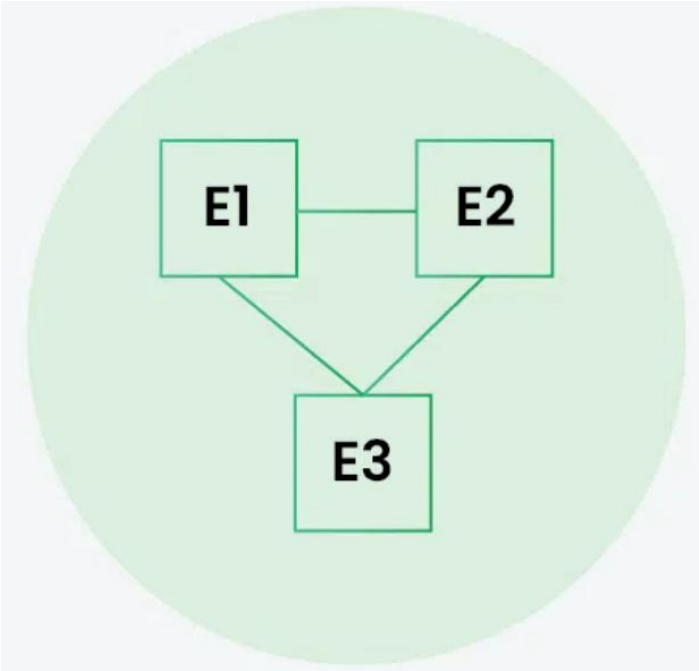
Ejemplo: Sistema de gestión de Pedidos

- Una pequeña empresa de comercio electrónico desea implementar un sistema simple para gestionar pedidos de clientes.

  Un programador desarrolló rápidamente una solución en C++ implementando todo en una sola clase llamada OrderSystem

- El programa funciona ok,… sin embargo desde el punto de vista de diseño de software, el código es un desastre!

Cohesión (Cohesion)



- **Definición**: Mide qué tan relacionadas están las responsabilidades dentro de un módulo.
- Objetivo: Alta cohesión
- **Principio clave**: Una clase debe tener una sola responsabilidad.

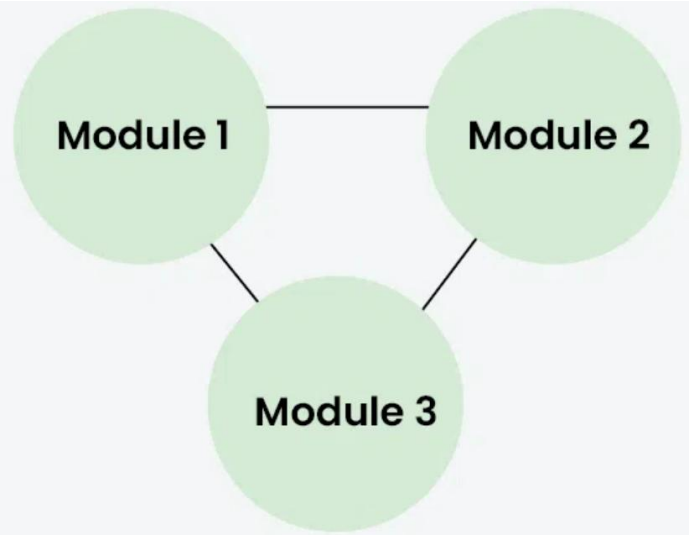
Ejemplo de Cohesión

- Baja cohesión: Una clase que hace muchas cosas diferentes.

- Alta cohesión: Una clase cuyas funciones están fuertemente relacionadas.
  Resultado: Código más claro, más fácil de mantener y probar.

- **En nuestro ejemplo inicial**, identificamos la clase Order, logrando mejorar la cohesión.

Acoplamiento (Coupling)



- **Definición**: Mide el grado de dependencia entre módulos o componentes del sistema.

- Objetivo: Bajo acoplamiento

- Ventajas:
  - Facilita mantenimiento
  - Permite cambiar componentes sin afectar todo el sistema

Ejemplo de Acoplamiento

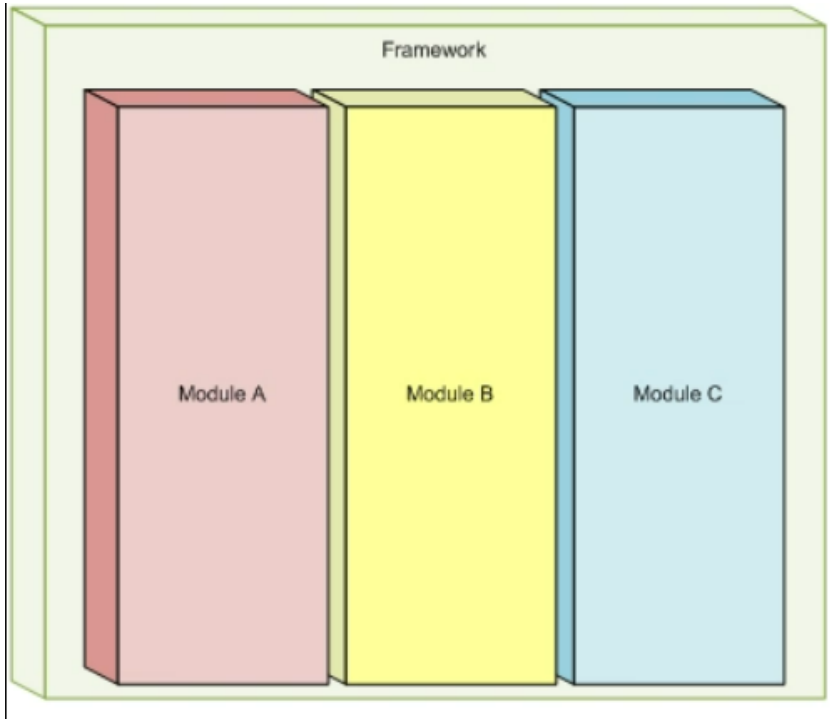
- Alto acoplamiento: Una clase depende directamente de otra implementación.
  Problema: Si cambia un módulo, debemos modificar otros.

- Buen diseño: debe reducir dependencias (bajo acoplamiento)

- **En nuestro ejemplo inicial:** reducimos el acoplamiento definiendo: Order, OrderCalculator, OrderPrinter,

Separación of Concerns (SoC)

- **Definición**: Dividir el sistema en partes que de manera que cada una de ellas se encargue de un problema específico



- En nuestro ejemplo inicial: Organizamos el sistema en capas claras:
  model → datos del dominio
  services → lógica del negocio
  presentation → salida al usuario
  infrastructure → persistencia

- Ejemplo 2: En aplicaciones web
  Fronted → interfaz del usuario
  Backend → lógica del negocio
  BD → almacenamiento

Conclusión

- Un buen diseño de software busca:
  - ✓ Alta cohesión
  - ✓ Bajo acoplamiento
  - ✓ Separación de responsabilidades

- Software mantenible, escalable y extensible.